U S E R M A N U A L

GraphRAG-Powered Cyber Threat Intelligence Platform

CTI Workstation — installation, operation and administration

Item	Value
Version	1.0 (Review III), October 2026
Audience	SOC analysts, CTI leads, platform operators, and engineers integrating firewalls, EDR or SIEM
Authors	Shivansh Sharma (23BCE0254) Himali Sharma (23BCE0317) Aayush Sinha (23BCE0558)
Institution / guide	VIT Vellore · Dr. Lydia Jane G
Source	github.com/Shivansharmaaaa/graphrag

Contents

Section	Page
1. About This Manual	3
2. System Overview	3
3. Installation	5
4. Signing In	6
5. Using the Console	7
6. Walkthroughs	14
7. Enforcement API for Security Tools	15
8. Configuration Reference	16
9. Administration and Maintenance	16
10. Troubleshooting	17
11. Glossary	17

1. About This Manual

This manual explains how to install the CTI Workstation, use each page of the analyst console, connect enforcement tools, and keep the platform healthy. Design rationale and measurements are in the separate Final Report.

If you are…	Read
an analyst	Sections 4, 5 and 6
a CTI lead or detection engineer	also 5.4 (blocklist), 5.6 (ingest) and 7
a platform operator	Sections 3, 8, 9 and 10
integrating a firewall, EDR or SIEM	Section 7

Conventions. Commands are shown in a monospace box and run from the project folder (`nosql/`). Text in `monospace` is typed exactly. Screens are shown in the console's dark theme.

2. System Overview

The workstation stores cyber threat intelligence in four databases, each used for what it does best, and answers questions in plain English using all four at once.

Store	Holds	You see it in
Neo4j	the threat graph: actors, campaigns, malware, infrastructure, techniques, victims, indicators, reports	Graph explorer; graph hops in GraphRAG answers
MongoDB	full reports, STIX bundles, users, audit log, ingest log	Report browser; documents in answers; Audit
Redis	the live IOC blocklist, each entry expiring after its TTL	IOC blocklist; IOC state in answers
Qdrant	report passages as vectors for similarity search	Semantic search; passages in answers

GraphRAG answers. When you ask a question, the system retrieves passages, graph paths, reports and IOC states, and a local language model writes the answer. Every sentence must cite evidence; sentences without a valid citation are removed, and if too few remain the answer is replaced by an evidence-only summary or a refusal.

Roles and clearance. Each account has a role that limits what it can change and a TLP clearance that limits what it can read.

Role	Can	Cannot
analyst	ask questions, browse the graph and reports, search, view the blocklist and own audit entries	publish IOCs, ingest, run admin actions

Role	Can	Cannot
lead	everything an analyst can, plus publish and revoke IOCs and ingest reports or STIX bundles	re-seed, reindex, delete reports
admin	everything, including re-seed, vector reindex, report deletion and all users' audit entries	—

TLP	Meaning for this platform (FIRST TLP 2.0)
WHITE / CLEAR	no restriction
GREEN	community use; visible to GREEN clearance and above
AMBER	limited distribution; hidden from GREEN accounts (list, search and GraphRAG), detail page returns 403
RED	named recipients only; visible to RED clearance

3. Installation

3.1 Requirements

Component	Requirement
Operating system	macOS, Linux, or Windows with WSL 2
Docker	Docker Desktop or Docker Engine with Compose v2; at least 4 GB of memory for the containers
Python	3.10 or newer (3.11 tested)
Node.js	18 or newer, for the console
Ollama (optional)	for fluent answers; without it the platform answers in retrieve-only mode
Ports	5173 (console), 8000 (API), 7474/7687 (Neo4j), 27017 (MongoDB), 6379 (Redis), 6333 (Qdrant), 11434 (Ollama)

3.2 Option A — run the API and console locally

```
cp .env.example .env
docker compose up -d            # Neo4j, MongoDB, Redis, Qdrant (wait ~30 s)
python3 -m venv .venv && .venv/bin/pip install -r backend/requirements.txt
make api                        # terminal 1: API on http://localhost:8000
make web                        # terminal 2: console on http://localhost:5173
```

On its first start the API creates the database schema and loads the seed corpus into all four stores; this takes a few seconds. API documentation is served at `http://localhost:8000/docs`.

3.3 Option B — everything in containers

```
cp .env.example .env
docker compose --profile app up -d --build      # stores + API + console on :5173
```

To stop: `docker compose --profile app stop`. Stop the containerised API and console before using Option A, because both use ports 8000 and 5173.

3.4 Local language model

```
ollama pull llama3.2:3b         # default model, meets the 15 s answer target
ollama pull llama3.1             # optional: fuller answers, slower (set
OLLAMA_MODEL=llama3.1)
```

The `llm` dot in the console sidebar turns green when Ollama is reachable and the configured model is pulled. In containers, the API reaches Ollama on the host through `host.docker.internal`; to run Ollama in a container instead, add `--profile llm` and set `OLLAMA_URL_DOCKER=http://ollama:11434` in `.env`.

3.5 Check the installation

```
curl -s http://localhost:8000/api/health     # "startup": {"ok": true, "detail":
"ready"}
make test                                    # 16 unit tests, no databases needed
make test-integration                        # 15 API tests against the running stores
```

4. Signing In

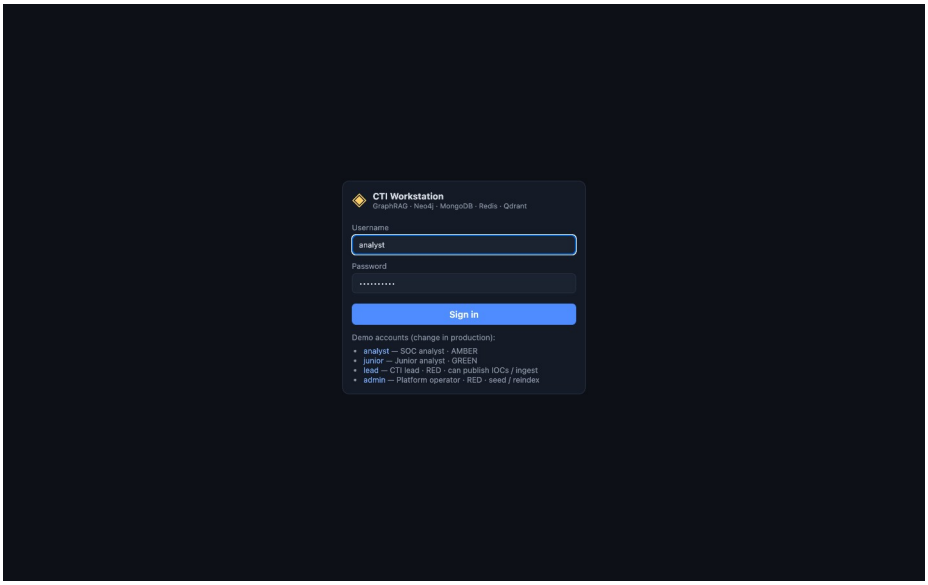


Figure 1. Sign-in page with the demo accounts.

Account / password	Role	Clearance	Use it to
analyst / analyst123	analyst	AMBER	explore and ask questions
junior / junior123	analyst	GREEN	see TLP filtering (AMBER hidden)
lead / lead123	lead	RED	publish IOCs and ingest
admin / admin123	admin	RED	re-seed, reindex, delete reports

Click an account name to fill the form, or type the credentials and press **Sign in**. A session lasts eight hours; when it expires you are returned to this page. The sidebar shows your name, role and clearance, the health of each store, and **Sign out**.

> **Change these passwords and the session secret before using the platform outside a lab** (`JWT_SECRET` in `.env`). The demo accounts are created automatically when the user collection is empty.

5. Using the Console

5.1 GraphRAG console

Type a question, or click one of the six example questions, and press **Ask**. **hops** (1–3) sets how far the graph is expanded from the entities named in the question, **top-k chunks** how many passages are retrieved, and **use local LLM** switches between a written answer and an evidence-only (retrieve-only) answer.



Figure 2. A cited answer with a commodity-tool caution and the evidence panel.

Element	Meaning
mode badge	llm (written by the model) or retrieve-only (evidence sentences; used when the LLM is off, down, or its answer failed the citation check)
TLP badge	the highest TLP among the reports the answer cites
citation coverage	share of the model's sentences that carried a valid citation; below 50% the answer is refused
citation chips	G = graph edge, C = passage, R = report, I = IOC state; click one to highlight it in the evidence panel
⚠ caution	shown when a tool used by many actors (for example Mimikatz) appears: it is weak evidence of attribution
removed sentences	expand to see sentences the groundedness gate dropped
audit link	opens the stored record of this answer for replay

The evidence panel has five tabs: **Graph hops** (an animated graph and the numbered facts), **Chunks** (passages with similarity scores and a 2-D similarity map), **Documents** (reports used, with their

report-specific fields), **IOC state** (live or expired on the blocklist, with remaining TTL) and **Latency** (time spent in each store and the model).

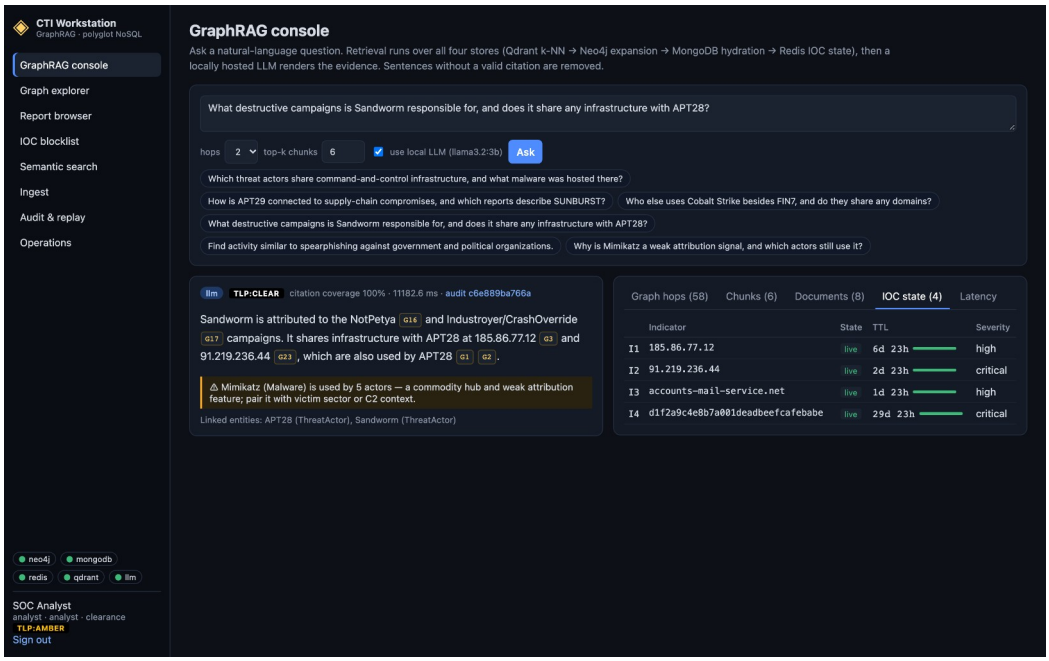


Figure 3. IOC state tab: whether each indicator in the evidence is live on the blocklist.

5.2 Graph explorer

Shows the whole graph (reports hidden). Search by name, alias, id or IOC value; click a node to open its neighbourhood; run the presets — shared infrastructure, shared tooling, common victims, tool users — or find the shortest path between two ids. Traversals stop at commodity hubs so that tools such as Cobalt Strike do not swamp the view.

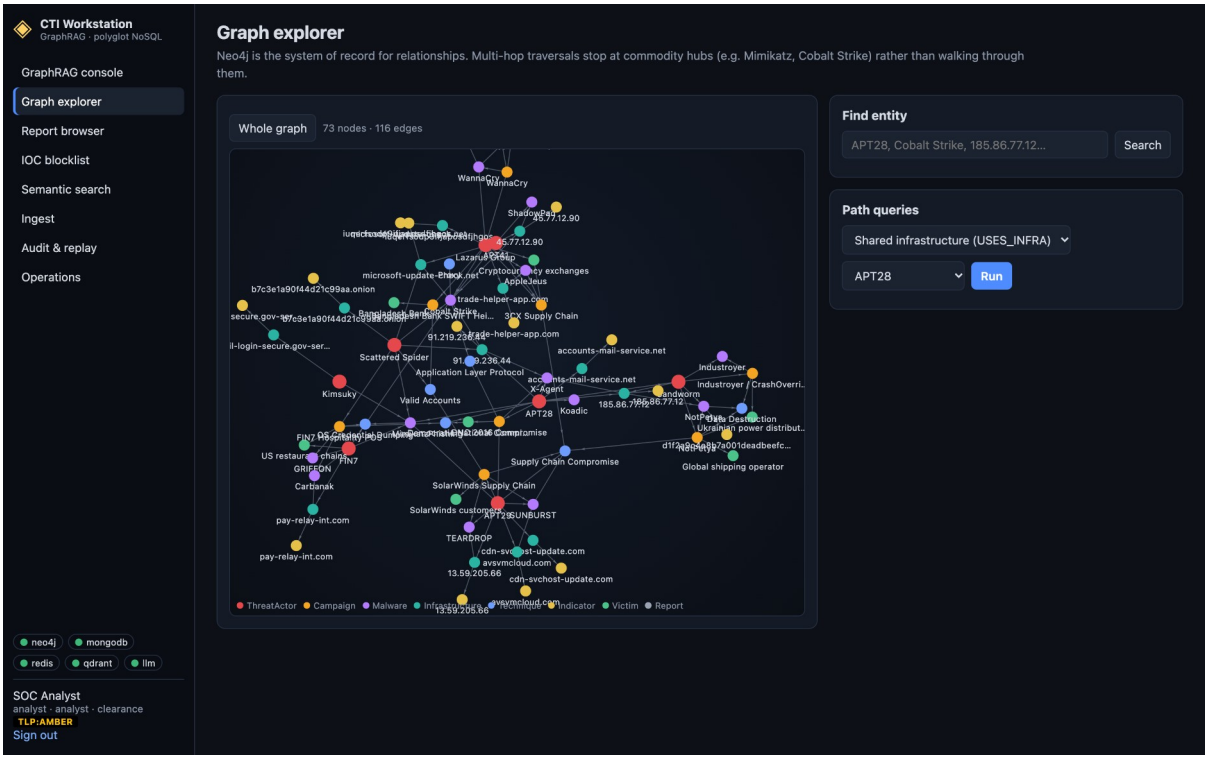


Figure 4. Graph explorer.

5.3 Report browser

Lists the reports your clearance allows, newest first, with their report-specific fields; filter by text, actor id or TLP. Opening a report shows the full body, linked actors, campaigns, malware and IOCs. An administrator can delete a report, which also removes its passages and graph links. Accounts without clearance see a notice of how many reports are hidden, and opening one returns “withheld by TLP policy”.

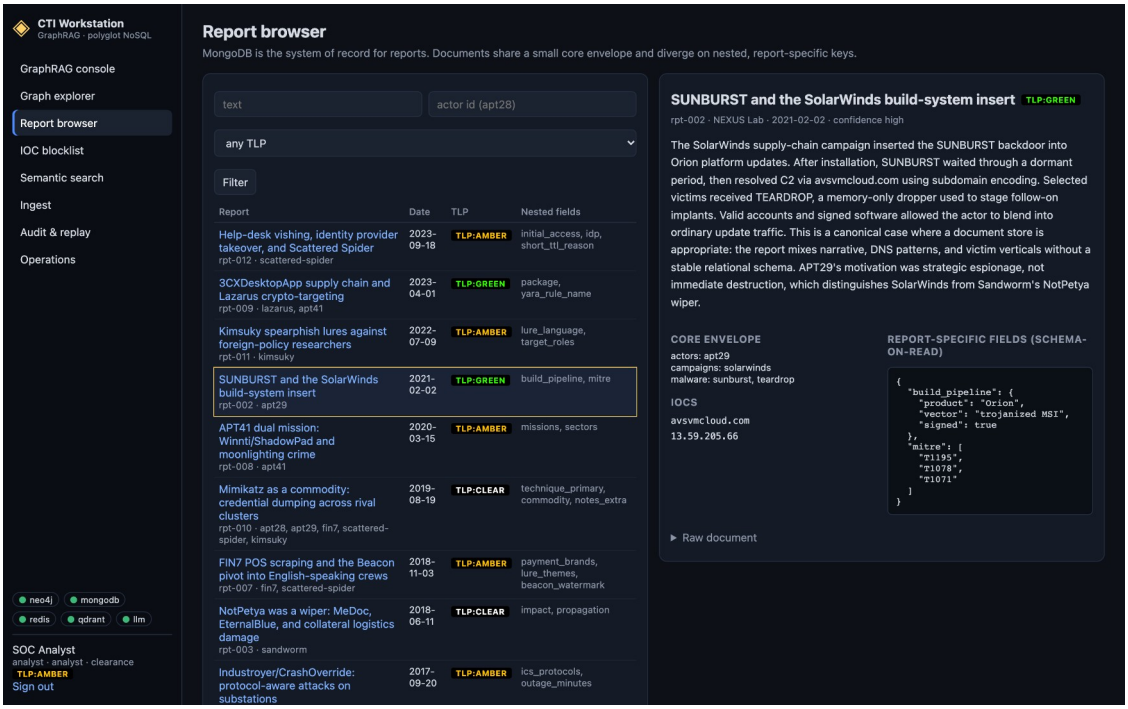


Figure 5. Report detail, including fields specific to this report (build pipeline, MITRE techniques).

5.4 IOC blocklist

The hot set lists live indicators with severity, actor or campaign, publisher and a ticking TTL. When a TTL reaches zero the indicator moves to **Recently expired**: it is no longer enforced, but **show historical graph presence** proves the graph still records it. **O(1) lookup** checks a single value the way a firewall would.

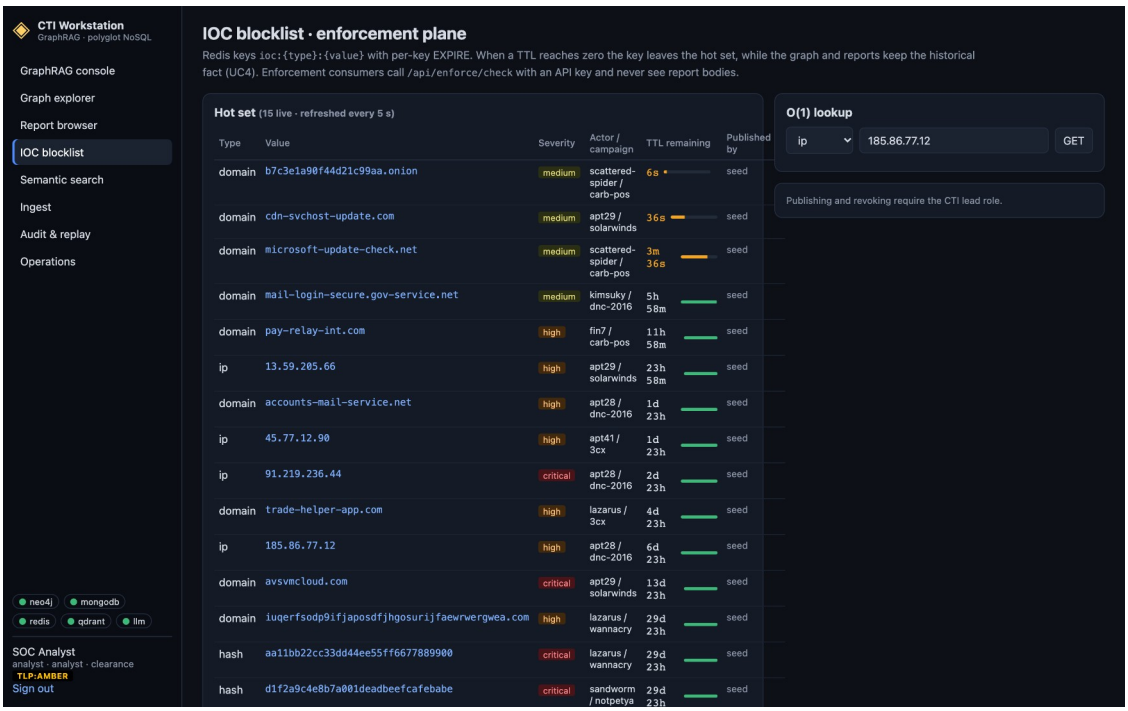


Figure 6. IOC blocklist with live TTLs.

Publishing (lead or admin). Fill in type (ip, domain, hash, url), value, TTL in seconds (up to 90 days), severity and optionally actor and campaign, then press **SET … EX**. **revoke** removes an entry immediately. Neither action changes the historical graph.

5.5 Semantic search

Finds passages that mean something similar to your text, not just matching words. Filter by actor ids, TLP levels and a date range; results show the score, report, TLP and passage, plus a similarity map. Use it for questions like “what else looks like this lure?”; use the graph explorer for “who is connected to whom?”.

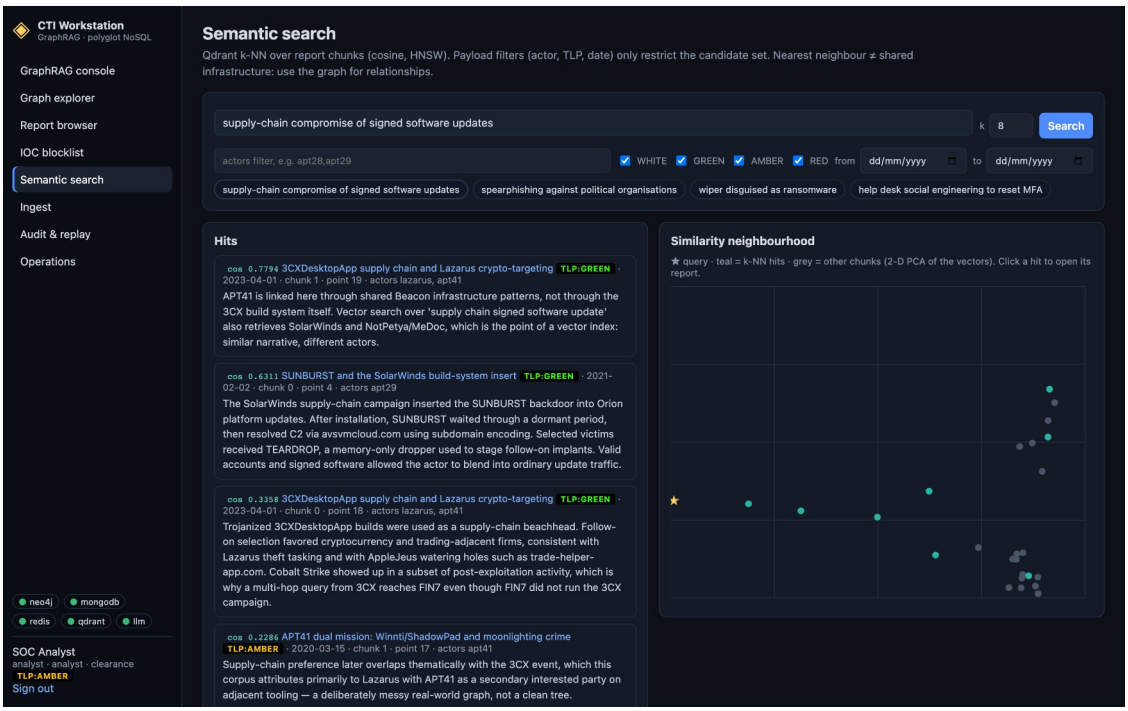


Figure 7. Semantic search.

5.6 Ingest (lead or admin)

Free-text report: enter title, body, source, date, TLP and optional report-specific fields as JSON. Entities and IOCs are found automatically, and defanged IOCs (`hxxp`, `[.]`) are restored. **STIX 2.1 bundle:** choose a `.json` file or paste a bundle; TLP is read from marking definitions, and `valid_until` becomes the blocklist TTL if **publish indicators** is ticked.

Run ingest job writes to Neo4j, MongoDB, Qdrant and (if requested) Redis, showing each step's status and time. If a step fails, fix the cause and press **retry**: the job resumes from the first step that did not finish.

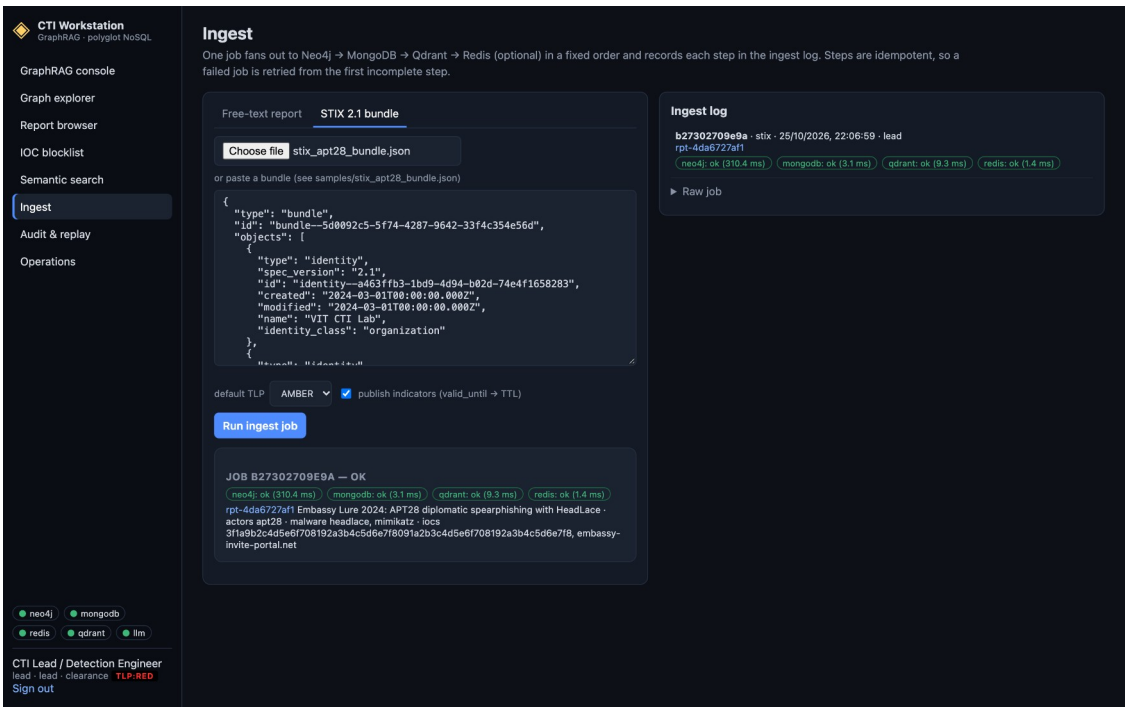


Figure 8. Ingesting the sample STIX bundle: each store's step and the resulting report.

5.7 Audit and replay

Every answer is recorded with the hops, passages, reports and IOC states it used. Select an entry and press **Replay against the stores** to check each item still exists and whether any IOC has changed state since — for example, an indicator that has expired.

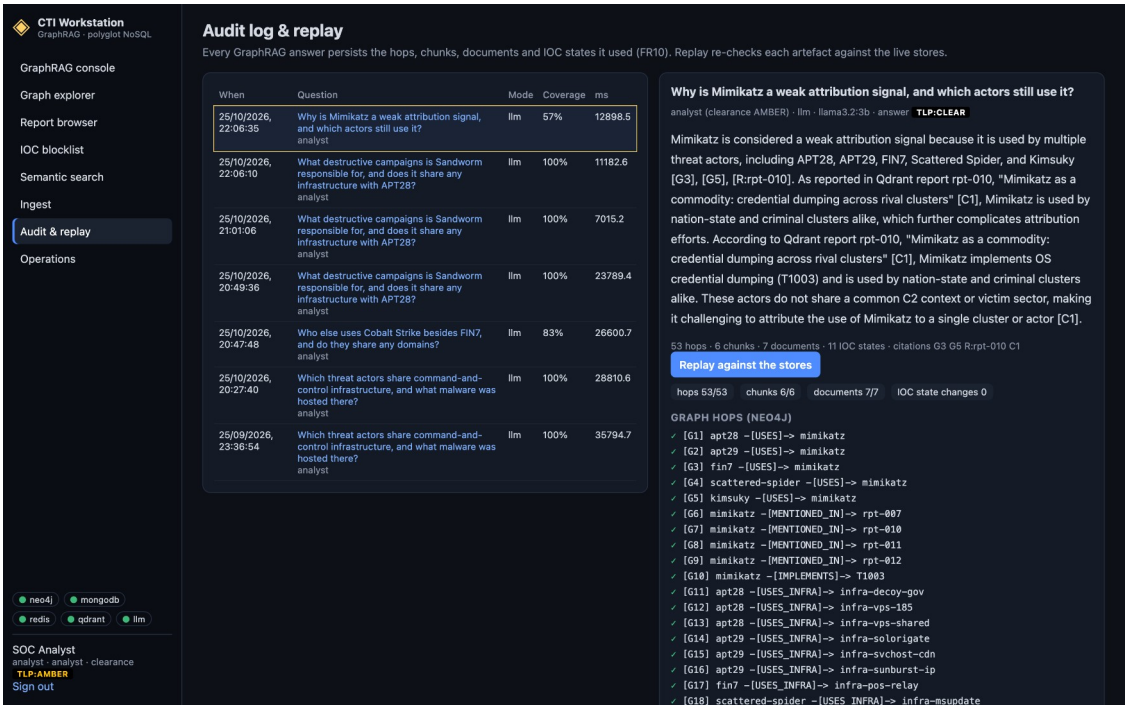


Figure 9. Audit entry replayed against the live stores.

5.8 Operations

Shows each store's status and round-trip time, latency percentiles per store against their targets (Redis p99 < 5 ms, Neo4j p95 < 500 ms, GraphRAG p95 < 15 s), the graph inventory and the cross-store

integrity check. Administrators can **re-seed** (reload the initial corpus; removes ingested data) and **rebuild the vector index**.

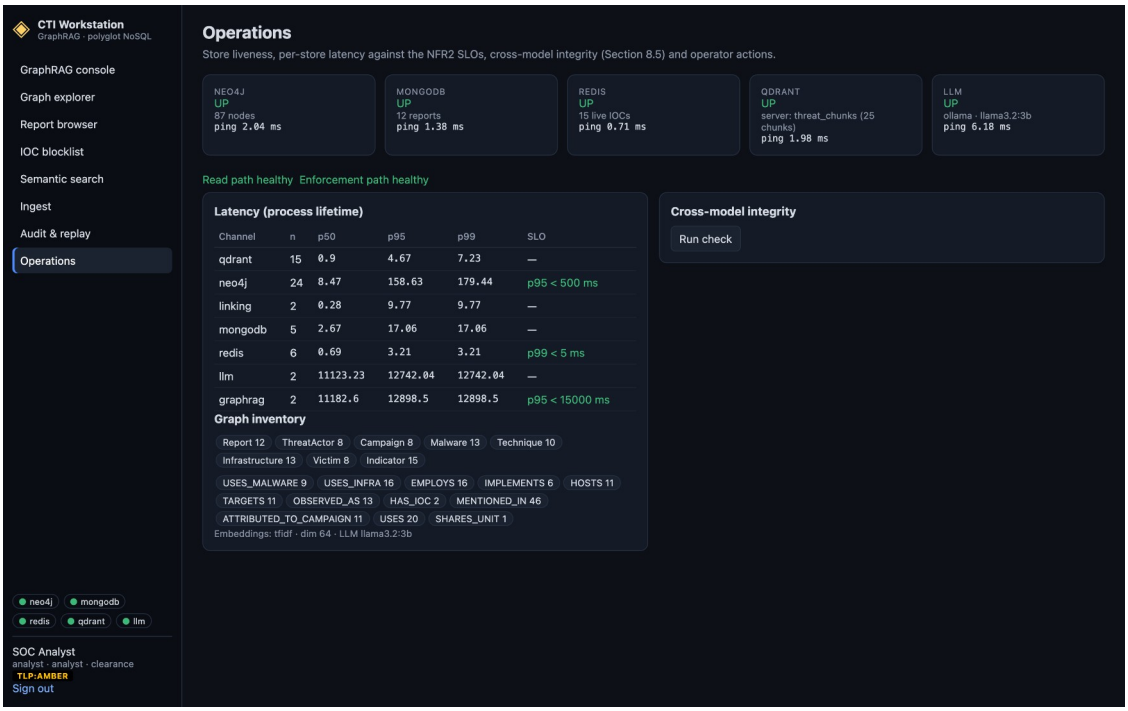


Figure 10. Operations: store health, latency against SLOs and integrity.

6. Walkthroughs

The walkthroughs below are the use cases the platform was designed for. They work on a freshly seeded stack (`make seed`).

6.1 Shared infrastructure (UC1)

1. Sign in as **analyst** and open the GraphRAG console.
2. Ask: “Which actors share C2 infrastructure with APT28?”
3. The answer names Sandworm (shared VPS 185.86.77.12) and Scattered Spider (91.219.236.44) and cites report rpt-001.
4. Open **IOC state** to see whether those IPs are live on the blocklist.

6.2 Similar activity (UC2)

1. Ask: “Find activity similar to spearphishing against government and political organizations.”
2. The **Chunks** tab shows the DNC passages first; the graph expands to APT28 and APT29; **Documents** lists the full reports.

6.3 Commodity tools (UC3)

1. Ask: “Why is Mimikatz a weak attribution signal, and which actors still use it?”
2. The answer lists five actors and shows the ⚠ caution: a tool shared by many actors is weak evidence of who is behind an intrusion.

6.4 Indicator expiry (UC4)

1. Re-seed (`make seed`), then open the **IOC blocklist** within a minute.
2. Three seeded IOCs have 90 s, 120 s and 300 s TTLs; watch them count down and move to **Recently expired**.
3. Click **show historical graph presence**: the graph still records the indicator although it is no longer enforced.

6.5 TLP filtering

1. Sign out and sign in as **junior** (GREEN).
2. The report browser hides AMBER reports and shows how many are hidden.
3. Open `/reports/rpt-001`: the report is withheld. Ask the UC1 question again: AMBER reports are not used and the answer is labelled GREEN or lower.

6.6 Ingest a STIX bundle

1. Sign in as **lead**, open **Ingest → STIX 2.1 bundle** and choose `samples/stix_apt28_bundle.json`.
2. Tick **publish indicators** and press **Run ingest job**; all four steps turn green.

3. Ask “diplomatic invitation lures against foreign ministries”: the new report (Embassy Lure 2024) is retrieved.

7. Enforcement API for Security Tools

Firewalls, EDR and SIEM tools use a separate API that reads only the Redis blocklist. They authenticate with the `X-API-Key` header (`CONSUMER_API_KEY` in `.env`), not with a user account.

```
curl -H "X-API-Key: edr-consumer-key" \
     "http://localhost:8000/api/enforce/check?type=ip&value=185.86.77.12"
{"blocked": true, "ttl_remaining": 604010, "severity": "high"}

curl -H "X-API-Key: edr-consumer-key" "http://localhost:8000/api/enforce/export?
format=txt"
ip,185.86.77.12
domain,avsvmcloud.com
```

Situation	Response
indicator live	200, blocked: true, remaining TTL and severity
indicator absent or expired	200, blocked: false — never looked up in any other database
wrong or missing key	401
Redis unavailable	503 with blocked: true and fail_closed: true — treat as block

8. Configuration Reference

Settings are read from `.env` in the project folder (copy `.env.example`). Environment variables override the file.

Setting	Default	Purpose
NEO4J_URI / USER / PASSWORD	bolt://localhost:7687, neo4j, ctiplatform	graph store
MONGO_URI / MONGO_DB	mongodb://localhost:27017, cti	document store
REDIS_URL	redis://localhost:6379/0	blocklist
QDRANT_HOST / QDRANT_PORT	localhost, 6333	vector store (embedded fallback if unreachable)
OLLAMA_URL / OLLAMA_MODEL	http://localhost:11434, llama3.2:3b	local language model
LLM_TIMEOUT_S	60	give up on the model after this many seconds (retrieve-only answer)
EMBEDDER / OLLAMA_EMBED_MODEL	tfidf, nomic-embed-text	offline TF-IDF embeddings, or a neural model via Ollama
JWT_SECRET	change-me…	signs sessions — change outside a lab
CONSUMER_API_KEY	edr-consumer-key	key for the enforcement API — change outside a lab
CORS_ORIGINS	http://localhost:5173,…	browser origins allowed to call the API
MAX_HOPS / HUB_DEGREE_THRESHOLD / MAX_NEIGHBOURS_PER_NODE	3, 5, 12	graph expansion limits and when a tool counts as a commodity hub
VECTOR_TOP_K / MIN_CITATION_COVERAGE	6, 0.5	passages per question; refusal threshold
LLM_MAX_FACTS / DOCS / DOC_CHARS / MAX_IOCS	30, 6, 300, 12	how much evidence the model sees (more = slower, fuller)
LLM_NUM_PREDICT / LLM_KEEP_ALIVE	320, 30m	answer length cap; how long the model stays loaded
AUTO_SEED	true	load the seed corpus when the stores are empty
DATA_DIR	backend/data	fitted embedder and embedded-Qdrant files

9. Administration and Maintenance

Task	How
Restore the demo data and fresh TTLs	`make seed` or Operations → re-seed (removes ingested data)
Rebuild the vector index	Operations → rebuild vector index, or POST /api/admin/reindex (needed after changing EMBEDDER)
Restore the snapshots	`./scripts/restore_databases.sh dumps` (seed mode: `… seed`)
Run the tests	`make test`; with the stores running, `make test-integration`
Check answer quality and SLOs	`make eval-check` (fails if a gold fact or report is missed)
Benchmark	`docker compose --profile app stop`, then `make bench` (uses its own throwaway stores)
Change the model	`ollama pull <model>`, set OLLAMA_MODEL, restart the API
Logs	`docker logs cti-api`, or the terminal running `make api`

Task	How
Stop everything	`docker compose --profile app down` (data volumes are kept; add `-v` to delete them)

10. Troubleshooting

Symptom	Likely cause	Fix
Console shows a red neo4j dot after start	Neo4j needs ~30 s to start	wait; check `docker compose ps` until healthy
llm dot red, answers say retrieve-only	Ollama not running or model not pulled (the tag must match exactly)	start Ollama; `ollama pull llama3.2:3b`
Answers are slow (> 15 s)	a large model, or the model was unloaded	use llama3.2:3b; keep LLM_KEEP_ALIVE; lower LLM_MAX_* settings
"Insufficient evidence…"	the evidence did not support any cited sentence	rephrase with entity names, increase hops or top-k
Blocklist almost empty	seeded TTLs expired (days later)	`make seed`
Port 8000 or 5173 in use	both the containerised and local app are running	`docker compose --profile app stop`, or stop `make api` / `make web`
403 when opening a report	report TLP above your clearance	sign in with a higher clearance
Enforcement API returns 503	Redis is down — the API fails closed	restart Redis (`docker compose up -d redis`)
Semantic search returns nothing	vector collection missing	Operations → rebuild vector index
Storage folder … already accessed	two processes using embedded Qdrant	start the Qdrant container, or run a single API process

11. Glossary

- **CTI —** cyber threat intelligence: knowledge about threat actors, their tools, infrastructure and behaviour
- **IOC —** indicator of compromise: an IP, domain, URL or file hash associated with malicious activity
- **TTL —** time to live: seconds until a blocklist entry expires
- **TLP —** Traffic Light Protocol: sharing restriction on a report (WHITE/CLEAR, GREEN, AMBER, RED)
- **STIX 2.1 —** standard JSON format for exchanging CTI objects and their relationships
- **ATT&CK —** MITRE's catalogue of adversary techniques (for example T1566 Phishing)
- **GraphRAG —** answering questions with evidence retrieved from a graph and a vector index before generation
- **Commodity hub —** a tool used by many unrelated actors, and therefore weak evidence of attribution
- **Retrieve-only —** an answer made of evidence sentences without the language model
- **Groundedness gate —** the check that removes uncited sentences and refuses unsupported answers

- **Hot set —** the IOCs currently live in the Redis blocklist